GROUP 1 : Ferdia Comerford, Hani Ali, Zhuohan Dai

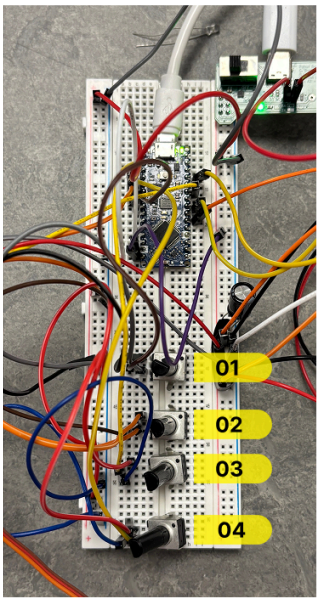
GITHUB LINK - **ZhuohanDai/Robot-Arm-and-Machine-Learning-Group-1**

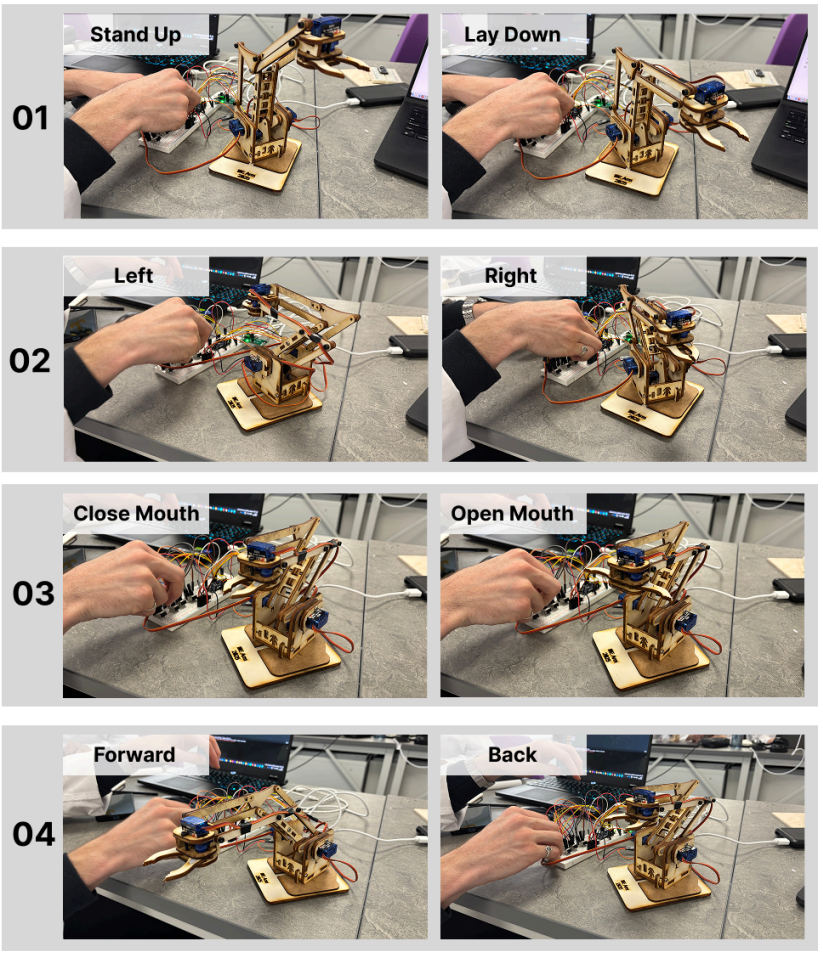
A1 - BUILT ROBOT ARM AND SYSTEM UNIT TESTS CHECK-IN

D1

DIGITAL CONTROL

1. Single unit test (Arduino sketch) for validating actuator output





2. Debugging action list

1. Check Wiring — Ensure all wires are securely connected to the correct pins.

   Expected observation: All connections are secure, with no loose or misplaced wires.

2. [BLINK CODE] Upload Blink Test — Upload BLINK program to verify the Arduino is linked correctly to Processing and Wekinator. If on board LED light responds to mouse movement the connection is correct. Check Wekinator input (mouse movement) is changing the classification in processing.

   Expected observation: The LED blinks at regular intervals.

3. Test Potentiometer — Upload Potentiometer code to Ardunino. Rotate each potentiometer and observe the corresponding input readings or servo response.

   Expected observation: The readings change smoothly as the potentiometer rotates, and the corresponding servo responds accordingly.

4. Check Power Supply — Verify that the Arduino and servos receive the appropriate power supply.

   Expected observation: The system powers on reliably, and the servos operate without unexpected resets or power interruptions.

5. Double-check System Functionality — Test all components together and check that each input produces the intended output.

   Expected observation: All four servos respond smoothly and independently to their corresponding potentiometers, with no unexpected movements or interruptions.

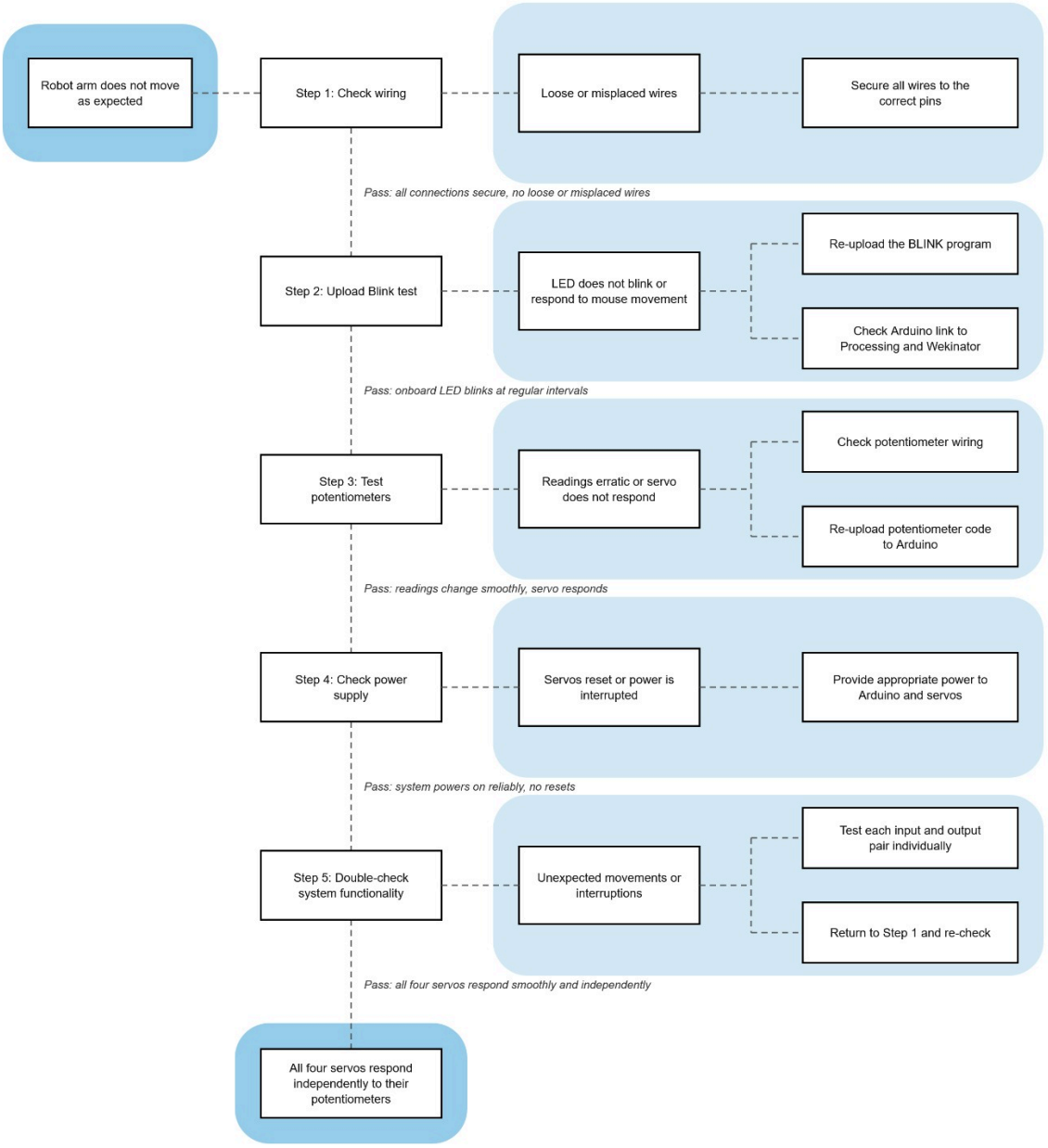
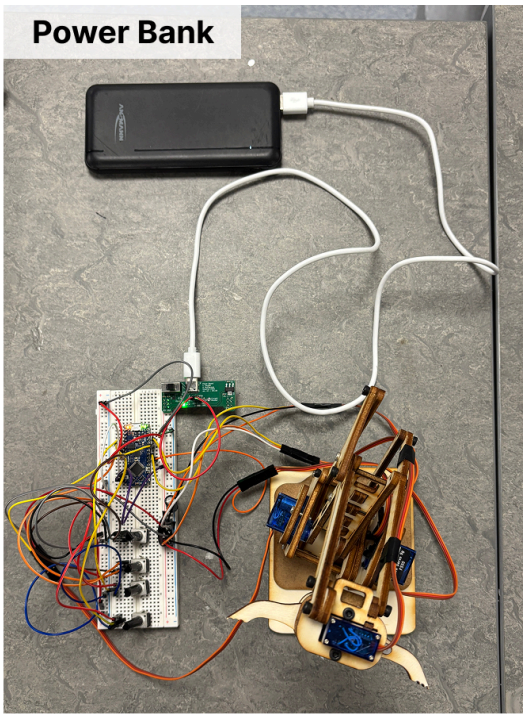
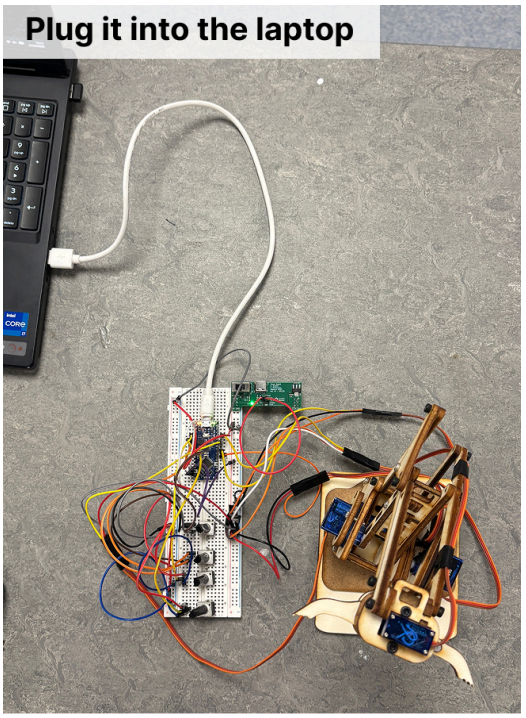


Figure 1. Decision tree for debugging

ELECTRONICS DESIGN

- Circuit powered by power bank
- Arduino powered by laptop



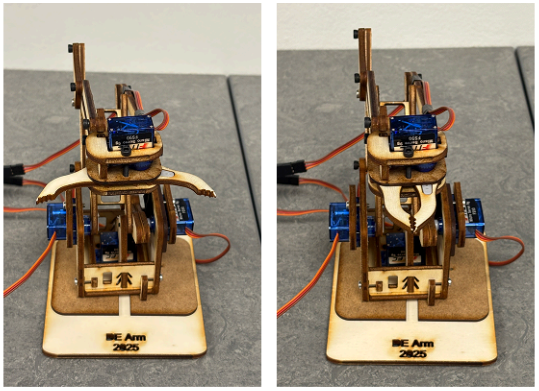


D2

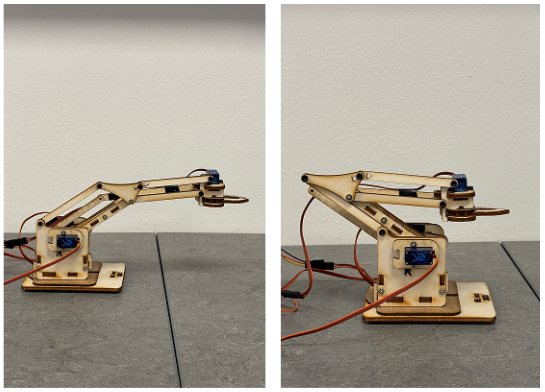
MECHANISM FUNCTIONING

1. **Independent actuation of all 4 servos over their full range of motion**

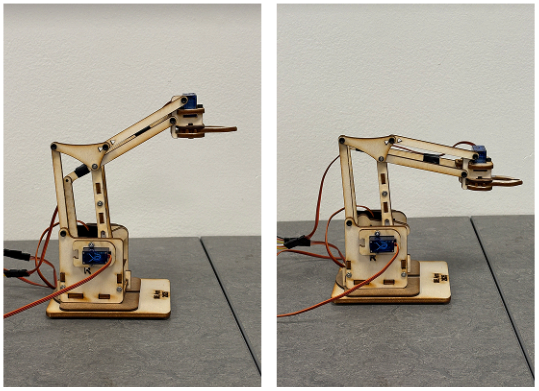
Mouth Open/ Close



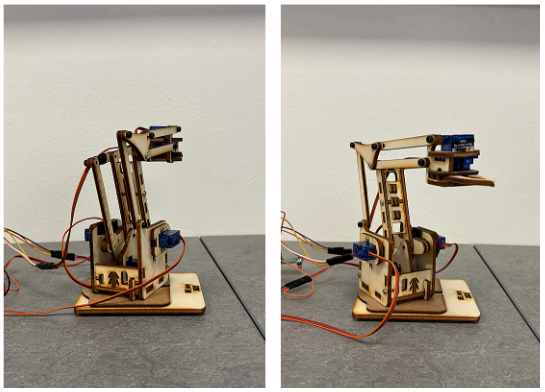
Forward/ Back



Stand Up/ Lay Down



Rotate Left/ Right

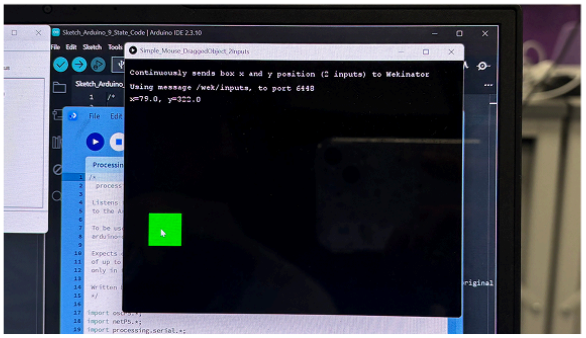


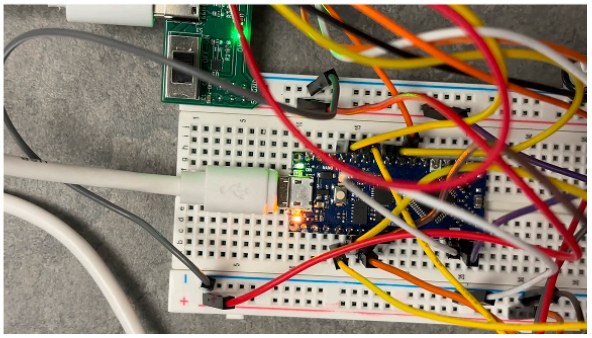
D3

MESSAGING FUNCTIONING

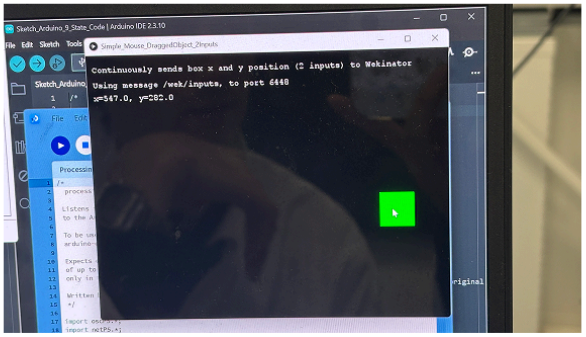
1. **Computer-to-Arduino Output Validation (Blink Test)**

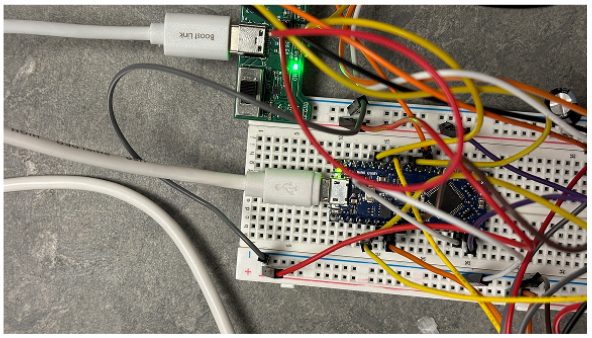
Blink quick





Blink slow





2. Debugging action list

1. Check Wiring — Ensure the Arduino is connected to the computer using a USB cable and that the onboard LED is working.

   Expected observation: The Arduino powers on, and the onboard LED responds when the program runs.

2. [BLINK CODE] Upload Arduino Test — Upload the blink code to the Arduino and check that the onboard LED flashes. The LED should blink using the default timing if no serial message is received.

   Expected observation: The onboard LED flashes on and off regularly.

3. [PROCESSING] Check Serial Port — Run Processing and check that `COM` matches the Arduino's actual port.

   Expected observation: Processing connects to the Arduino without serial port errors.

4. [WEKINATOR] Check Input and Classification — Start Wekinator and move the mouse or change the input. Check the Processing console to confirm that classification values are being received.

   Expected observation: The classification values change according to the input and appear in the Processing console.

5. [BLINK PATTERN] Test All Three Classes — Test classes 1 and 2. Each class should produce a different blink pattern.

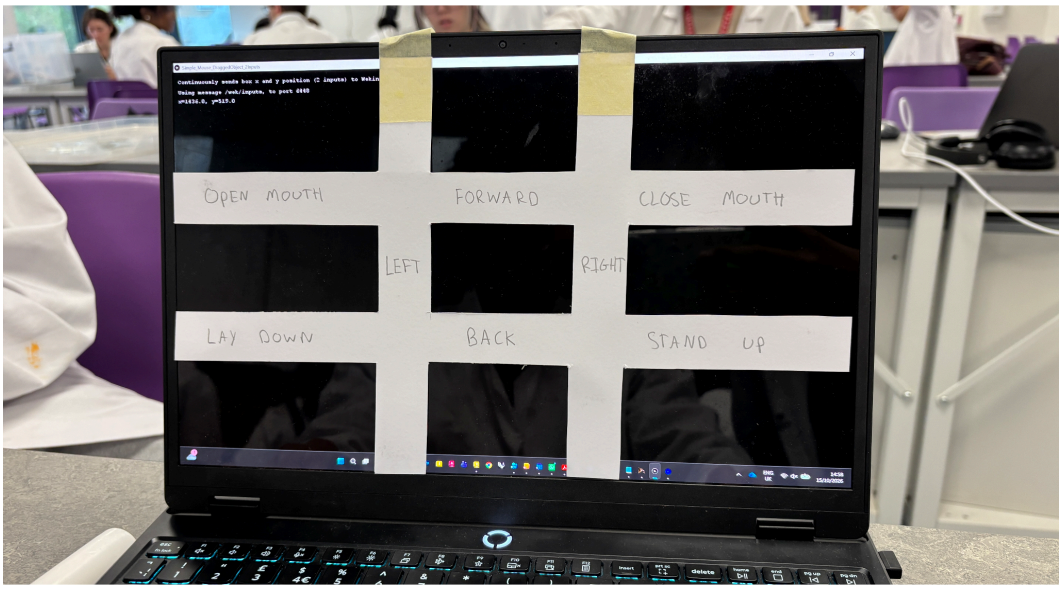
   Expected observation: The LED displays a distinct blink pattern for each class. If another value is received, the LED uses the default pattern of 1000 ms on and off.

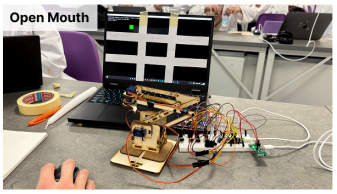
ARDUINO OUTPUT CONTROLLED BY WEKINATOR

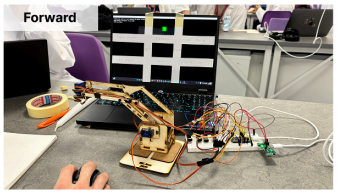
D4

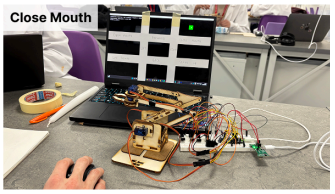
MACHINE LEARNING AND ARDUINO OUTPUT

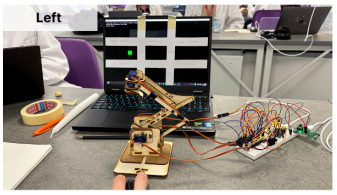
1. **Trained ML model running in Wekinator**

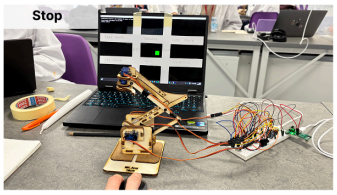


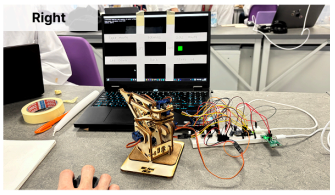


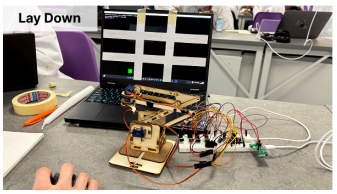


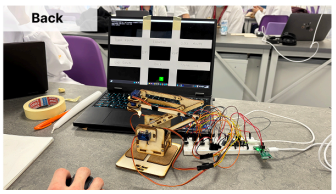


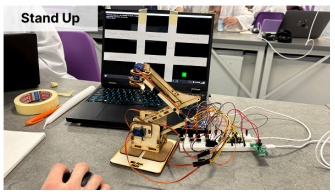












2. Debugging action list

1. Check Wiring — Ensure all four servos are connected to the correct Arduino pins: servo 1 to pin 9, servo 2 to pin 7, servo 3 to pin 5 and servo 4 to pin 3. Check that the power supply and ground connections are secure.

   Expected observation: All connections are secure, and the servos receive sufficient power without unexpected resets or interruptions.

2. [PROCESSING] Check Serial Connection — Run Processing and ensure that `COM` matches the Arduino's actual serial port.

   Expected observation: Processing connects to the Arduino successfully, with no serial port errors.

3. [WEKINATOR] Train and Test Nine Classifiers — Set up Wekinator with nine classes representing the intended robot arm movements. Train each class using the appropriate mouse movements or inputs, then test whether Wekinator can distinguish between them correctly.

   Expected observation: Wekinator recognises the nine classes and produces the correct classification number for each intended input.

4. [SERVO MOVEMENTS] Test All Four Servos — Test classes 1–8 individually. Classes 1 and 2 control servo 1, classes 3 and 4 control servo 2, classes 5 and 6 control servo 3, and classes 7 and 8 control servo 4. Check that each servo opens and closes as intended.

   Expected observation: Each servo responds to its assigned classes and moves smoothly between its programmed positions without unintended movements.

5. [CLASS 9] Test the No-Movement State — Send class 9 from Wekinator and check that the Arduino does not issue any new servo movement commands.

   Expected observation: All four servos remain at their existing positions when class 9 is detected.

6. Double-Check System Functionality — Run Wekinator, Processing and the Arduino together. Test all nine classifications repeatedly and check that Wekinator correctly distinguishes between inputs and that each class produces the intended response.

   Expected observation: The complete system reliably distinguishes between the nine classes, all four servos respond smoothly and independently to their assigned commands, and class 9 produces no new movement commands.

D1 - CODE - Potentiometer

Arduino:

```
/*

   Controls robot are with 4 potentiometer

   To be used to debug and to rule out servos / wiring issues if a
problem arises
   processing_osc_to_arduino.pde

   Written by Zhuohan, Hani, Ferdia  2026
*/


#include <Servo.h> // library for controlling servo motors

Servo servo1; // servo object for servo 1
Servo servo2; // servo object for servo 2
Servo servo3; // servo object for servo 3
Servo servo4; // servo object for servo 4

// Potentiometer pins
const int pot1 = A0; // potentiometer 1 is connected to analog pin A0
const int pot2 = A1; // potentiometer 2 is connected to analog pin A1
const int pot3 = A2; // potentiometer 3 is connected to analog pin A2
const int pot4 = A3; // potentiometer 4 is connected to analog pin A3

// Servo signal pins
const int servoPin1 = 9; // servo 1 signal wire is on pin 9
const int servoPin2 = 7; // servo 2 signal wire is on pin 7
const int servoPin3 = 5; // servo 3 signal wire is on pin 5
const int servoPin4 = 3; // servo 4 signal wire is on pin 3

void setup() {  // runs once when the Arduino starts or resets
  servo1.attach(servoPin1); // link servo 1 object to its pin
  servo2.attach(servoPin2); // link servo 2 object to its pin
  servo3.attach(servoPin3); // link servo 3 object to its pin
  servo4.attach(servoPin4); // link servo 4 object to its pin

  servo1.write(90); // start servo 1 at the middle position (90
degrees)
  servo2.write(90); // start servo 2 at the middle position (90
degrees)
```

```
  servo3.write(90); // start servo 3 at the middle position (90
degrees)
  servo4.write(90); // start servo 4 at the middle position (90
degrees)
}

void loop() { // runs over and over forever
  int val1 = analogRead(pot1); // read potentiometer 1 (a number from 0
to 1023)
  int val2 = analogRead(pot2); // read potentiometer 2 (a number from 0
to 1023)
  int val3 = analogRead(pot3); // read potentiometer 3 (a number from 0
to 1023)
  int val4 = analogRead(pot4); // read potentiometer 4 (a number from 0
to 1023)

  int angle1 = map(val1, 0, 1023, 0, 180);  // convert the 0-1023
reading into a 0-180 degree angle
  int angle2 = map(val2, 0, 1023, 0, 180); // same conversion for servo
2
  int angle3 = map(val3, 0, 1023, 0, 180); // same conversion for servo
3
  int angle4 = map(val4, 0, 1023, 0, 180); // same conversion for servo
4


  servo1.write(angle1); // move servo 1 to the angle set by
potentiometer 1
  servo2.write(angle2); // move servo 2 to the angle set by
potentiometer 2
  servo3.write(angle3); // move servo 3 to the angle set by
potentiometer 3
  servo4.write(angle4); // move servo 4 to the angle set by
potentiometer 4

  delay(15); // short pause so the servos have time to reach their
positions
}
```

D3 - CODE - Blink

Arduino:

```
/*
   arduino-osc-blink

   Listens for serial messages and controls the
   the on-board LED according to the message.

   To be used with the Processing sketch
   processing_osc_to_arduino.pde

   Written by Becky Stewart 2022
*/



// variable used for LED blink pattern
int blinkCode = -1;

void setup() {
  Serial.begin(9600);  // initialise the serial communication


  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  // wait for a message to arrive
  if (Serial.available()) {
    // save the message when it arrives
    blinkCode = Serial.read();
  }

  // create two variables to control the blink patter
  int onDelay;
  int offDelay;

  switch (blinkCode) {
    case 1:
      // on one second, off a quarter second
      onDelay = 100;
      offDelay = 100;
      break;
    case 2:
```

```
      // on a half second, off one second
      onDelay = 2000;
      offDelay = 2000;
      break;
    case 3:
      // on one and a half seconds, off 50 ms
      onDelay = 1500;
      offDelay = 50;
      break;
    default:
      // default to on one second, off one second
      onDelay = 1000;
      offDelay = 1000;
      break;
  }

  digitalWrite(LED_BUILTIN, HIGH);  // turn the LED on (HIGH is the
voltage level)
  delay(onDelay);                   // wait for the delayed time
  digitalWrite(LED_BUILTIN, LOW);   // turn the LED off by making the
voltage LOW
  delay(offDelay);                  // wait for the delayed time
}
```

Processing :

```
/*
 processing_osc_to_arduino

 Listens for OSC messages and sends a Serial message
 to the Arduino.

 To be used with the Arduino sketch
 arduino-osc-blink.ino

 Expects one output from Wekinator with a classifier
 of up to 3 classes. More than 3 classes will result
 only in the default blinking.

 Written by Becky Stewart 2022
 */

import oscP5.*;
import netP5.*;
import processing.serial.*;
```

```
OscP5 oscP5;
NetAddress myRemoteLocation;

Serial port;

int arduinoMessage = -1;

void setup() {
  size(256, 150);
  println("Available serial ports:");
  // print the list of available ports
  printArray(Serial.list());

  // Uses the printed list of ports to choose which number your board is connected to.
  // Change the below line Serial.list()[X] to your selected port, replacind X with
  // your number.
  // The last parameter (e.g. 9600) is the speed of the communication. It has to
  // correspond to the value passed to Serial.begin() in your Arduino sketch.
  port = new Serial(this, "COM9", 9600);

  // If you know the name of the port used by the Arduino board, you can specify
  // it directly like this.
  //port = new Serial(this, "COM1", 9600);

  // Start oscP5 and listen for incoming messages at port 12000
  oscP5 = new OscP5(this, 12000);
}


void draw() {
  // draw a gradient from black to white
  // it doesn't do anything, just to fill the window
  for (int i = 0; i < 256; i++) {
    stroke(i);
    line(i, 0, i, 150);
  }

  // write the current X-position of the mouse to the serial port
  // useful for debugging, may want to comment out when listening
  // for osc messages
  //int mouseMessage = (int)map(mouseX, 0, 256, 0, 4);
  //updateArduino(mouseMessage);

}


void oscEvent(OscMessage theOscMessage) {
```

```
  /* check if theOscMessage has the address pattern we are looking for. */

  if (theOscMessage.checkAddrPattern("/wek/outputs")==true) {
    /* check if the typetag is the right one. */
    if (theOscMessage.checkTypetag("f")) {
      /* parse theOscMessage and extract the values from the osc message arguments. */
      float firstValue = theOscMessage.get(0).floatValue();
      print("### received an osc message /test with typetag f.");
      println(" values: "+firstValue);

      updateArduino((int)firstValue);
      return;
    }
  }
  println("### received an osc message. with address pattern
"+theOscMessage.addrPattern());
}

// send a new message to the Arduino
void updateArduino(int newMessage) {
  // check that the message is different from the last one sent
  if ( newMessage != arduinoMessage ) {
    // if so then update the message
    arduinoMessage = newMessage;
    // print for debugging convenience
    println(arduinoMessage);
    // send the message to the Arduino
    port.write(arduinoMessage);
  }
}
```

D4 - CODE - Final

Arduino :

```
/*

   Controls robot arm with 9 states using mouse control on wekinator

   To be used with the Processing sketch
   processing_osc_to_arduino.pde

   Written by Zhuohan, Hani, Ferdia  2026
*/


#include <Servo.h> // include the Servo library so we can control servo
motors

Servo servo1;  // create a servo object for servo 1
Servo servo2;  // create a servo object for servo 2
Servo servo3;  // create a servo object for servo 3
Servo servo4;  // create a servo object for servo 4

// variable used to store the latest message from Processing (named
blinkCode from the original LED example)
int robotCode = -1;

void setup() {  // runs once when the Arduino starts or resets
  Serial.begin(9600);  // initialise the serial communication
servo1.attach(9); // tell the servo1 object it is wired to pin 9
servo2.attach(7); // tell the servo2 object it is wired to pin 7
servo3.attach(5); // tell the servo3 object it is wired to pin 5
servo4.attach(3); // tell the servo4 object it is wired to pin 3
  servo1.write(90); // move servo 1 to its 90 degree starting position
  servo2.write(90); // move servo 2 to its 90 degree starting position
  servo3.write(90); // move servo 3 to its 90 degree starting position
  servo4.write(90); // move servo 4 to its 90 degree starting position

  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {  // runs over and over forever
  // wait for a message to arrive
  if (Serial.available()) {
    // save the message when it arrives
```

```
    robotCode = Serial.read();  // read one byte (the state number 1-9)
and store it
  }

  switch (robotCode) {  // choose what to do based on the last message
received
    case 1: // when Classifier recieved from wekinator is 1 run below
code
      // Servo 1 Open
      servo1.write(30);  // turn servo 1 to 30 degrees (open)
      break;
    case 2: // when Classifier recieved from wekinator is 2  run below
code
      // Servo 1 Closed
     servo1.write(180);   // turn servo 1 to 180 degrees (closed)
     case 3: // when Classifier recieved from wekinator is 3 run below
code
      // Servo 2 Open
      servo2.write(30);   // turn servo 2 to 30 degrees (open)
      break;
    case 4: // when Classifier recieved from wekinator is 4 run below
code
      // Servo 2 Closed
     servo2.write(180);   // turn servo 2 to 180 degrees (closed)
     case 5: // when Classifier recieved from wekinator is 5 run below
code
      // Servo 3 Open
      servo3.write(30);   // turn servo 3 to 30 degrees (open)
      break;
    case 6: // when Classifier recieved from wekinator is 6 run below
code
      // Servo 3 Closed
     servo3.write(180);   // turn servo 3 to 180 degrees (closed)
     case 7: // when Classifier recieved from wekinator is 7 run below
code
      // Servo 4 Open
      servo4.write(1);   // turn servo 4 to 1 degrees (open)
      break;
    case 8: // when Classifier recieved from wekinator is 8 run below
code
      // Servo 4 Closed  // turn servo 4 to 180 degrees (closed)
     servo4.write(90);
      break;
```

```
    case 9:
       //Do nothing
      break;


  }

}
```

Processing:

```
/*
  processing_osc_to_arduino

 Listens for OSC messages and sends a Serial message
 to the Arduino.

 To be used with the Arduino sketch
 arduino-osc-blink.ino

 Expects one output from Wekinator with a classifier
 of up to 9 classes.

 Written by Becky Stewart 2022 - Modified by Hani, Zhuohan and Ferdia 2026
 */

import oscP5.*;
import netP5.*;
import processing.serial.*;

OscP5 oscP5;
NetAddress myRemoteLocation;

Serial port;

int arduinoMessage = -1;

void setup() {
  size(256, 150);
  println("Available serial ports:");
  // print the list of available ports
  printArray(Serial.list());

  // Uses the printed list of ports to choose which number your board is connected to.
  // Change the below line Serial.list()[X] to your selected port, replacind X with
  // your number.
  // The last parameter (e.g. 9600) is the speed of the communication. It has to
```

```
  // correspond to the value passed to Serial.begin() in your Arduino sketch.
  port = new Serial(this, "COM9", 9600);

  // If you know the name of the port used by the Arduino board, you can specify
  // it directly like this.
  //port = new Serial(this, "COM1", 9600);

  // Start oscP5 and listen for incoming messages at port 12000
  oscP5 = new OscP5(this, 12000);
}


void draw() {
  // draw a gradient from black to white
  // it doesn't do anything, just to fill the window
  for (int i = 0; i < 256; i++) {
    stroke(i);
    line(i, 0, i, 150);
  }

  // write the current X-position of the mouse to the serial port
  // useful for debugging, may want to comment out when listening
  // for osc messages
  //int mouseMessage = (int)map(mouseX, 0, 256, 0, 4);
  //updateArduino(mouseMessage);

}


void oscEvent(OscMessage theOscMessage) {
  /* check if theOscMessage has the address pattern we are looking for. */

  if (theOscMessage.checkAddrPattern("/wek/outputs")==true) {
    /* check if the typetag is the right one. */
    if (theOscMessage.checkTypetag("f")) {
      /* parse theOscMessage and extract the values from the osc message arguments. */
      float firstValue = theOscMessage.get(0).floatValue();
      print("### received an osc message /test with typetag f.");
      println(" values: "+firstValue);

      updateArduino((int)firstValue);
      return;
    }
  }
  println("### received an osc message. with address pattern
"+theOscMessage.addrPattern());
}
```

```
// send a new message to the Arduino
void updateArduino(int newMessage) {
  // check that the message is different from the last one sent
  if ( newMessage != arduinoMessage ) {
    // if so then update the message
    arduinoMessage = newMessage;
    // print for debugging convenience
    println(arduinoMessage);
    // send the message to the Arduino
    port.write(arduinoMessage);
  }
}
```

STATES

1 open mouth	**3** close mouth go back	**5** close mouth rotate left	**7** close mouth down
2 close mouth	**4** close mouth go forward	**6** close mouth rotate right	**8** close mouth up

1 (dodgy wire)	**4**	**2** (dodgy wire)
5	**9**	**6**
7	**3**	**8**